# PROJECT REPORT

# ON

# VOLUNTEER MANAGEMENT SYSTEM

# Table of Contents

# 1. ABSTRACT

This project details the development of a comprehensive web application designed to streamline and optimize volunteer management processes. The system addresses the critical need for efficient coordination of volunteers, events, and tasks within organizations. The application facilitates seamless communication, scheduling, and tracking of volunteers by providing a centralised platform. Key features include event creation and management, progress monitoring, and volunteer profile management. This application features a distinct portal system comprising an administrative portal, where the admin will register events for volunteers. The admin grants complete control over the system, enabling the admin to manage events, tasks, and volunteer profiles. The portal allows volunteers to view assigned tasks and events and track their contributions. This application aims to enhance organizational efficiency, improve volunteer engagement, and maximise the impact of volunteer efforts through intuitive and user-friendly web interfaces while maintaining robust administrative oversight.

# 2. INTRODUCTION

Many organizations rely on volunteers to achieve their goals. However, managing these volunteers' schedules and tasks can be challenging. This project aims to create a simple and effective web application to make volunteer management easier.

Imagine an organisation that hosts many events and needs help from volunteers. This application provides a central place where they can organise everything. It has one central part: an admin portal.

The admin section lets organizers create events, assign tasks to volunteers, and keep track of everyone's progress. They can also see all the information about the volunteers.

The admin section lets volunteers see what tasks they've been assigned and know when and where they need to be. This allows for flexible engagement and empowers volunteers to contribute based on their passions and availability.

This web application helps organizations and volunteers work together more smoothly, making contributing and managing volunteer efforts easier. It simplifies the process of organizing events and tasks and makes sure everyone is on the same page.

# 3. DESIGN

This design outlines the key features and structure of the Volunteer Management Web Application, focusing on user experience and functionality for both administrators and volunteers.

## 1. System Architecture

The platform follows a client-server architecture, where the front end interacts with the backend via RESTful APIs. Sequelize ORM is used for efficient database management.

- **Frontend:** React.js for a responsive and dynamic user interface.
- **Backend:** Node.js with Express to manage business logic and API interactions.
- **Database:** MySQL, managed using Sequelize for structured data handling.
- **Authentication:** JWT-based secure login and user session management.

## 2. Database Schema

The database consists of the following main tables:

- **Users:** Stores user credentials, roles, and authentication tokens.
- **Events:** Contains details (Name, Location, Description, Date & Time)
- **Volunteers:** Stores details of (Name, Email, Phone Number, Description)

## 3. Security Considerations

- **Password Hashing:** User passwords are securely hashed using bcrypt.
- **JWT Authentication:** Ensures safe user session management.
- **Role-Based Access Control:** Prevents unauthorised content modifications.

**VMS APP**

VMS-main/ → (Root directory of the Volunteer Management System)

|— api/ → (Backend: Node.js + Express)

| |— node_modules/ → (Stores installed dependencies)

| |— config/ → (Contains configuration files, e.g., database connection)

| | ├— config.js → (Database and environment configurations)

| |— controllers/ → (Handles request logic for different API endpoints)

| |— middleware/ → (Custom middleware functions for authentication, validation, etc.)

| |— migrations/ → (Database migration files)

```
|   |— models/  → (Sequelize models for database tables)
|   |   ├— comment.js  → (Comment model for handling user comments)
|   |   ├— index.js  → (Model index file that initializes all models)
|   |   ├— post.js  → (Post model for managing blog posts)
|   |   ├— user.js  → (User model storing user details and authentication info)
|   |— routes/  → (Defines API endpoints)
|   |   ├— authentication.js  → (Handles user authentication routes)
|   |   ├— login.js  → (User login-related API routes)
|   |   ├— posts.js  → (Routes for managing posts)
|   |   ├— users.js  → (User-related routes)
|   |— .env  → (Environment variables file)
|   |— app.js  → (Main entry point for the backend server)
|   |— package-lock.json  → (Dependency lock file for npm)
|   |— package.json  → (Project dependencies and scripts)
|— client/  → (Frontend: React.js)
|   |— node_modules/  → (Stores frontend dependencies)
|   |— public/  → (Static assets like HTML, images, and favicon)
|   |   ├— index.html  → (Main HTML file where React mounts)
|   |— src/  → (Source code for the React app)
|   |   ├— components/  → (Reusable UI components)
|   |   |   ├— Navbar.jsx  → (Navigation bar component)
|   |   ├— pages/  → (Main application pages)
|   |   |   ├— Home.jsx  → (Homepage displaying an overview)
|   |   |   ├— Login.jsx  → (User login page)
```

| | | ├— Register.jsx → (User registration page)

| | | ├— Write.jsx → (Page for writing and submitting content)

| | ├— App.js → (Main component that defines routing and structure)

| | ├— axios.js → (Handles API requests using Axios)

| | ├— ErrorBoundary.js → (Catches and handles UI errors)

| | ├— index.js → (Entry point for the React app)

| | ├— style.scss → (Global styles using SCSS)

| |— package-lock.json → (Dependency lock file for npm)

| |— package.json → (Project dependencies and scripts)

| |— README.md → (Frontend-specific documentation)

| |— yarn.lock → (Lock file for Yarn dependencies)

|— README.md → (Main project documentation explaining setup and usage)

|— .gitignore → (Specifies files to be ignored by Git)

# 4. IMPLEMENTATION

Backend: Located in the API folder.

Frontend: Located in the Client folder.

## Database Setup:

- Open your SQL CLI and create a database named **VMS**.
- The database schema is managed using **Sequelize**, so no manual SQL script execution is required.

## Backend Setup:

Navigate to the API folder on your terminal.

**Install dependencies:**

- ✓ Step1: npm install
- ✓ Step2: npx sequelize db: migrate
- ✓ Step3: node app.js
- ✓ Step 4: The backend will run on the port specified in the .env file.

**Environment Variables (.env file for backend):**

DB_USERNAME=<YOUR_DB_USERNAME>

DB_PASSWORD=<your_DB_PASSWORD>

DB_NAME=blog

DB_HOST=127.0.0.1

DB_DIALECT=mysql

NODE_ENV=development

JWT_SECRET=blogSecretKey or <ENTER_A_SECRET_KEY>

PORT=8080 or <ENTER_A_SPECIFIC_PORT>

## Frontend Setup:

Open another terminal and navigate to the client folder.

**Install dependencies:** npm install

If using a port other than 8080, update axios.js in /client/src to reflect the correct backend URL.

**Start the frontend server:** npm start

## CODES

### Index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./style.scss";

const root =
ReactDOM.createRoot(document.getElement
ById("root"));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

### App.js

```
import {
  createBrowserRouter,
  RouterProvider,
  Outlet,
} from "react-router-dom";
import Register from "./pages/Register";
import Login from "./pages/Login";
import Home from "./pages/Home";
import Write from "./pages/Write";
import Navbar from "./components/Navbar";

import "./style.scss";

const Layout = () => {
  return (
    <>
      <Navbar/>
        <Outlet/>

    </>
);
};

const router = createBrowserRouter([
  {
    path: "/",
    element: <Layout/>,
    children:[
    {
      path:"/",
      element:<Home/>
    },

    {
      path:"/Write",
      element:<Write/>
  },
  ]
  },
  {
    path: "/home",
    element: (
      <div>
        <Navbar />
        <Home />
      </div>
    ),
  },
  {
    path: "/login",
    element: <Login />,
```

```
  },
  {
    path: "/register",
    element: <Register />,
  },
  {
    path: "/write",
    element: <Write />,
  },
], {
  future: {
    v7_startTransition: true,
    v7_relativeSplatPath: true,
  },
});

function App() {
  return (
    <div className="App">
      <div className="container">
      <RouterProvider router={router} />
      </div>
    </div>
  );
}

export default App;
```

## Home.js

```
import { useState, useEffect } from "react";
import raxios from "../axios";

const Home = () => {
  const [events, setEvents] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchEvents = async () => {
      try {
        const response = await
raxios.get("/posts");
        setEvents(response.data);
      } catch (error) {
        console.error("Error fetching events:",
error.response?.data || error);
      } finally {
        setLoading(false);
      }
    };
    fetchEvents();
  }, []);

  // Function to remove all HTML tags from
event content
  const stripHtmlTags = (html) => {
    return html.replace(/<\/?[^>]+(>|$)/g, ""); //
Removes all HTML tags
  };

  return (
    <div style={styles.container}>


      {loading ? (
        <p style={styles.loadingText}>Loading
events...</p>
      ) : events.length > 0 ? (
        <div style={styles.eventGrid}>
```

```
        {events.map((event) => (
          <div key={event.id}
style={styles.eventCard}>
            <div style={styles.eventHeader}>
              <h3
style={styles.eventTitle}>{event.title}</h3>

            </div>
            <p
style={styles.eventContent}>{stripHtmlTags(
event.content)}</p>
          </div>
        ))}
      </div>
    ) : (
      <p style={styles.noEvents}>🚀 No
events available.</p>
    )}
  </div>
 );
};

// **Styles**
const styles = {
 container: {
  maxWidth: "1000px",
  margin: "0 auto",
  padding: "20px",
  fontFamily: "'Poppins', sans-serif",
  textAlign: "center",
 },
 header: {
  fontSize: "32px",
  fontWeight: "bold",
  color: "#333",
  marginBottom: "20px",
 },
 eventGrid: {
  display: "grid",
  gridTemplateColumns: "repeat(auto-fit,
minmax(300px, 1fr))",
  gap: "20px",
 },
 eventCard: {
  background: "linear-gradient(135deg,
#667eea, #764ba2)",
  padding: "20px",
  borderRadius: "12px",
  boxShadow: "0px 4px 10px rgba(0, 0, 0,
0.1)",
  color: "#fff",
  textAlign: "left",
  transition: "transform 0.3s ease-in-out",
 },
 eventTitle: {
  fontSize: "20px",
  fontWeight: "bold",
 },
 eventContent: {
  fontSize: "16px",
  lineHeight: "1.5",
  marginTop: "10px",
 },
 eventDate: {
  fontWeight: "bold",
  background: "rgba(255, 255, 255, 0.2)",
  padding: "5px 10px",
```

```
    borderRadius: "8px",
  },
  loadingText: {
    fontSize: "18px",
    color: "#777",
  },
  noEvents: {
    fontSize: "18px",
    color: "#ff5e62",
    fontWeight: "bold",
  },
};

export default Home;
```

## Login.js

```
import React, { useState } from "react";
import { Link, useNavigate } from "react-
router-dom";
import raxios from "../axios";

const Login = () => {
  const [inputs, setInputs] = useState({
    identifier: "",
    password: "",
  });

  const [error, setError] = useState(null);
  const navigate = useNavigate();

  const handleChange = (e) => {
    setInputs((prev) => ({ ...prev,
[e.target.name]: e.target.value }));
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError(null);

    try {
      const res = await raxios.post("/login",
inputs);

      if (res.data?.token) {
        sessionStorage.setItem("token",
res.data.token);
        navigate("/");
      } else {
        setError("Invalid response from
server. Please try again");
      }
    } catch (err) {
      console.error("Login error:", err);
      setError(err.response?.data?.message ||
"Something went wrong. Please try again.");
    }
  };

  return (
    <>
      <style>
        {`
        .auth {
          display: flex;
          justify-content: center;
          align-items: center;
          height: 100vh;
```

```
        background: linear-
gradient(135deg, #1e3c72, #2a5298);
    }

    .auth-container {
        background: white;
        padding: 40px;
        border-radius: 10px;
        box-shadow: 0 4px 10px rgba(0,
0, 0, 0.2);
        text-align: center;
        width: 350px;
    }

    .auth h1 {
        font-size: 24px;
        color: #333;
        margin-bottom: 20px;
    }

    .auth form {
        display: flex;
        flex-direction: column;
        gap: 15px;
    }

    .auth input {
        width: 100%;
        padding: 12px;
        border: 1px solid #ccc;
        border-radius: 8px;
        font-size: 16px;
        outline: none;
    }

    .auth input:focus {
        border-color: #1e3c72;
        box-shadow: 0 0 5px rgba(30, 60,
114, 0.5);
    }

    .auth button {
        background: #1e3c72;
        color: white;
        font-size: 18px;
        padding: 12px;
        border: none;
        border-radius: 8px;
        cursor: pointer;
    }

    .auth button:hover {
        background: #2a5298;
    }

    .auth .error {
        color: red;
        font-size: 14px;
        margin-top: 10px;
    }

    .auth span {
        display: block;
        margin-top: 15px;
        font-size: 14px;
    }

    .auth span a {
```

```
            color: #1e3c72;
            text-decoration: none;
            font-weight: bold;
        }

        .auth span a:hover {
            color: #2a5298;
        }

        /* Responsive */
        @media (max-width: 768px) {
            .auth-container {
                width: 90%;
            }
        }
        `}
      </style>

      <div className="auth">
        <div className="auth-container">
          <h1>Login</h1>
          <form
onSubmit={handleSubmit}>
            <input
              type="text"
              placeholder="Email or
Username"
              name="identifier"
              value={inputs.identifier}
              onChange={handleChange}
              required
            />
            <input
              type="password"
              placeholder="Password"
              name="password"
              value={inputs.password}
              onChange={handleChange}
              required
            />
            <button
type="submit">Login</button>

            {error && <p
className="error">{error}</p>}

            <span>
              Don't have an account?
<Link to="/register">Register</Link>
            </span>
          </form>
        </div>
      </div>
    </>
  );
};

export default Login;
```

## Register.js

```
import React, { useState } from "react";
import { Link } from "react-router-dom";
import raxios from "../axios";

const Register = () => {
  const [inputs, setInputs] = useState({
    username: "",
```

```
    email: "",
    password: "",
  });

  const [error, setError] = useState(null);
  const [success, setSuccess] = useState(null);

  const handleChange = (e) => {
    setInputs((prev) => ({ ...prev,
[e.target.name]: e.target.value }));
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError(null);
    setSuccess(null);

    try {
      const res = await
raxios.post("/register", inputs);
      setSuccess(res.data.message);
    } catch (err) {
      setError(err.response?.data?.message ||
"Something went wrong!");
    }
  };

  return (
    <div className="auth">
      <div className="auth-container">
        <h1>Register</h1>
        <form onSubmit={handleSubmit}>
          <input
            required
            type="text"
            placeholder="Username"
            name="username"
            value={inputs.username}
            onChange={handleChange}
          />
          <input
            required
            type="email"
            placeholder="E-mail"
            name="email"
            value={inputs.email}
            onChange={handleChange}
          />
          <input
            required
            type="password"
            placeholder="Password"
            name="password"
            value={inputs.password}
            onChange={handleChange}
          />
          <button
type="submit">Register</button>

          {error && <p
className="error">{error}</p>}
          {success && <p
className="success">{success}</p>}

          <span>
            Already have an account?
<Link to="/login">Login</Link>
          </span>
```

```
            </form>
          </div>
        </div>
    );
};

export default Register;
```

## Events.js

```
import React, { useState } from "react";
import { useNavigate } from "react-router-dom";
import raxios from "../axios";
import ReactQuill from "react-quill";
import "react-quill/dist/quill.snow.css";

const Write = () => {
  const [title, setTitle] = useState("");
  const [content, setContent] = useState("");
  const [date, setDate] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  // Volunteer Form States
  const [volunteerName, setVolunteerName] = useState("");
  const [volunteerEmail, setVolunteerEmail] = useState("");
  const [volunteerPhone, setVolunteerPhone] = useState("");
  const [volunteerMessage, setVolunteerMessage] = useState("");
  const [volunteers, setVolunteers] = useState([]);

  const navigate = useNavigate();

  const handlePublish = async () => {
    if (!title || !content || !date) {
      alert("Title, content, and date are required!");
      return;
    }

    setLoading(true);
    setError(null);

    try {
      const token = sessionStorage.getItem("token");
      if (!token) {
        throw new Error("User is not authenticated. No token found.");
      }

      const response = await raxios.post(
        "/posts",
        { title, content, date, userId: 1 },
        {
          headers: {
            "Content-Type": "application/json",
            Authorization: `Bearer ${token}`,
          },
        }
      );

      if (response.status === 201) {
        alert("Event Added successfully!");
```

```
      } else {
        throw new Error("Unexpected response
status: " + response.status);
      }
    } catch (err) {
      console.error("Error:", err);
      setError(err.response?.data?.message ||
"Failed to Add Event!");
    }

    setLoading(false);
  };

  // Handle Volunteer Form Submission
  const handleVolunteerSubmit = (e) => {
    e.preventDefault();

    if (!volunteerName || !volunteerEmail ||
!volunteerPhone || !volunteerMessage) {
      alert("All volunteer fields are required!");
      return;
    }

    const newVolunteer = {
      id: Date.now(),
      name: volunteerName,
      email: volunteerEmail,
      phone: volunteerPhone,
      message: volunteerMessage,
    };

    setVolunteers([...volunteers,
newVolunteer]);
    alert("volunteer Added!");

    // Reset form
    setVolunteerName("");
    setVolunteerEmail("");
    setVolunteerPhone("");
    setVolunteerMessage("");
  };

  return (
    <div style={styles.container}>
      {/* Content Section */}
      <div style={styles.contentSection}>
        <input
          type="text"
          placeholder="Enter your title here..."
          value={title}
          onChange={(e) =>
setTitle(e.target.value)}
          style={styles.input}
        />
        <div style={styles.editorContainer}>
          <ReactQuill style={styles.editor}
theme="snow" value={content}
onChange={setContent} />
        </div>
      </div>

      {/* Sidebar */}
      <div style={styles.sidebar}>
        {/* Post Section */}
        <div style={styles.card}>
          <h2 style={styles.heading}>Post</h2>
```

```
        <label style={styles.label}>📅 Select
Date:</label>
        <input
          type="date"
          value={date}
          onChange={(e) =>
setDate(e.target.value)}
          style={styles.datePicker}
        />

        <button style={styles.publishButton}
onClick={handlePublish}
disabled={loading}>
          {loading ? "Adding..." : "Add Event"}
        </button>
        {error && <p
style={styles.error}>{error}</p>}
      </div>

      {/* Volunteer Sign-Up Form */}
      <div style={styles.volunteerCard}>
        <h2 style={styles.heading}>Volunteer
Registration</h2>
        <form
onSubmit={handleVolunteerSubmit}>
          <input
            type="text"
            placeholder="Full Name"
            value={volunteerName}
            onChange={(e) =>
setVolunteerName(e.target.value)}
            style={styles.input}
            required
          />
          <input
            type="email"
            placeholder="Email Address"
            value={volunteerEmail}
            onChange={(e) =>
setVolunteerEmail(e.target.value)}
            style={styles.input}
            required
          />
          <input
            type="tel"
            placeholder="Phone Number"
            value={volunteerPhone}
            onChange={(e) =>
setVolunteerPhone(e.target.value)}
            style={styles.input}
            required
          />
          <textarea
            placeholder="Event id"
            value={volunteerMessage}
            onChange={(e) =>
setVolunteerMessage(e.target.value)}
            style={styles.textarea}
            required
          />
          <button type="submit"
style={styles.volunteerButton}>
            Register
          </button>
        </form>
      </div>
```

```
      {/* Registered Volunteers */}
      {volunteers.length > 0 && (
        <div style={styles.volunteerList}>
          <h2 style={styles.heading}>Registered
Volunteers</h2>
          {volunteers.map((volunteer) => (
            <div key={volunteer.id}
style={styles.volunteerItem}>
              <p><strong>{volunteer.name}</stro
ng></p>
              <p>{volunteer.email} |
{volunteer.phone}</p>
              <p
style={styles.volunteerMessage}>"{volunteer.
message}"</p>
            </div>
          ))}
        </div>
      )}
    </div>
  </div>
 );
};

// **Styles**
const styles = {
 container: {
  display: "flex",
  gap: "30px",
  padding: "40px",
  backgroundColor: "#f8f9fc",
  minHeight: "100vh",
 },
 contentSection: {
  flex: 3,
  padding: "20px",
  backgroundColor: "#fff",
  borderRadius: "10px",
  boxShadow: "0 4px 10px rgba(0, 0, 0, 0.1)",
 },
 sidebar: {
  flex: 1,
  display: "flex",
  flexDirection: "column",
  gap: "20px",
 },
 input: {
  padding: "12px",
  fontSize: "16px",
  borderRadius: "8px",
  outline: "none",
  width: "100%",
  border: "1px solid #ccc",
  transition: "0.3s",
 },
 textarea: {
  padding: "12px",
  fontSize: "16px",
  borderRadius: "8px",
  outline: "none",
  width: "100%",
  height: "100px",
  resize: "none",
 },
 volunteerButton: {
  padding: "12px",
  borderRadius: "6px",
  backgroundColor: "#007bff",
```

```
    color: "#fff",
    cursor: "pointer",
    fontSize: "16px",
    width: "100%",
    transition: "0.3s",
  },
  publishButton: {
    padding: "10px",
    borderRadius: "6px",
    backgroundColor: "#007bff",
    color: "#fff",
    cursor: "pointer",
    fontSize: "16px",
    width: "100%",
    transition: "0.3s",
    marginTop: "20px",
  },
    datePicker:{
      padding: "12px",
      fontSize: "16px",
      borderRadius: "8px",
      outline: "none",
      width: "100%",
      border: "1px solid #ccc",
      transition: "0.3s",
    }
};

export default Write;
```

## Navbar.js

```
import React from "react";
import { Link, useNavigate } from "react-router-dom";

const Navbar = () => {
  const navigate = useNavigate();
  const username = sessionStorage.getItem("username");

  const handleLogout = () => {
    sessionStorage.clear();
    navigate("/login");
  };

  return (
    <div className="Navbar">
      <div className="container">
        <div className="links">
          {username ? (
            <>
              <span className="see">
                <Link className="link" to="/single">Profile</Link>
              </span>
              <span className="see logout" onClick={handleLogout}>Logout</span>
              <span className="see">
                <Link className="link" to="/write">Write</Link>
              </span>
            </>
          ) : (
            <>
              <span className="see">
```

```
            <Link className="link"
to="/Home">Home</Link>
            </span>
            <span className="see">
              <Link className="link"
to="/Write">Event Register</Link>
            </span>

            <span className="see">
              <Link className="link"
to="/Login">Login</Link>
            </span>
            <span className="see">
              <Link className="link"
to="/Register">Register</Link>
            </span>
          </>
        )}
      </div>
    </div>
  </div>
 );
};

export default Navbar;
```

## axios.js

```
import axios from "axios";

const raxios = axios.create(
  {
    baseURL: "http://localhost:8080"
  }
);

export default raxios;
```

## ErrorBoudary.js

```
import React from "react";

class ErrorBoundary extends
React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  componentDidCatch(error, info) {
    console.error("Error caught in Error
Boundary:", error, info);
  }

  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong.
Please try again later.</h1>;
    }
    return this.props.children;
  }
}

export default ErrorBoundary;
```

**.env**

```
DB_USERNAME=root
DB_PASSWORD=root
DB_NAME=blog
DB_HOST=127.0.0.1
DB_DIALECT=mysql
NODE_ENV=development
JWT_SECRET=blogSecretKey
PORT=8080
```

## Backend - App.js

```
const express = require('express');
const cors = require('cors');
const app = express();
require('express-async-errors');

app.use(cors({
   origin: 'http://localhost:3000',
   credentials: true
}));

const login = require('./routes/login');
const users = require('./routes/users');
const posts = require('./routes/posts');

app.use(express.json());
app.use('/login', login);
app.use('/', users);
app.use('/posts', posts);

const PORT = process.env.PORT || 8080;

app.listen(PORT, () => {
   console.log(`Started backend on port
${PORT}`);
});
```

# 5. RESULTS

This project successfully developed a distinct portal Volunteer Management Web Application.

The system provides:

- **Event Management:**
  - Create and delete events.
  - Set event dates, times, locations, and descriptions.
  - Manage event registration.
- **Volunteer Management:**
  - View and manage volunteer profiles.
  - Assign events to volunteers.

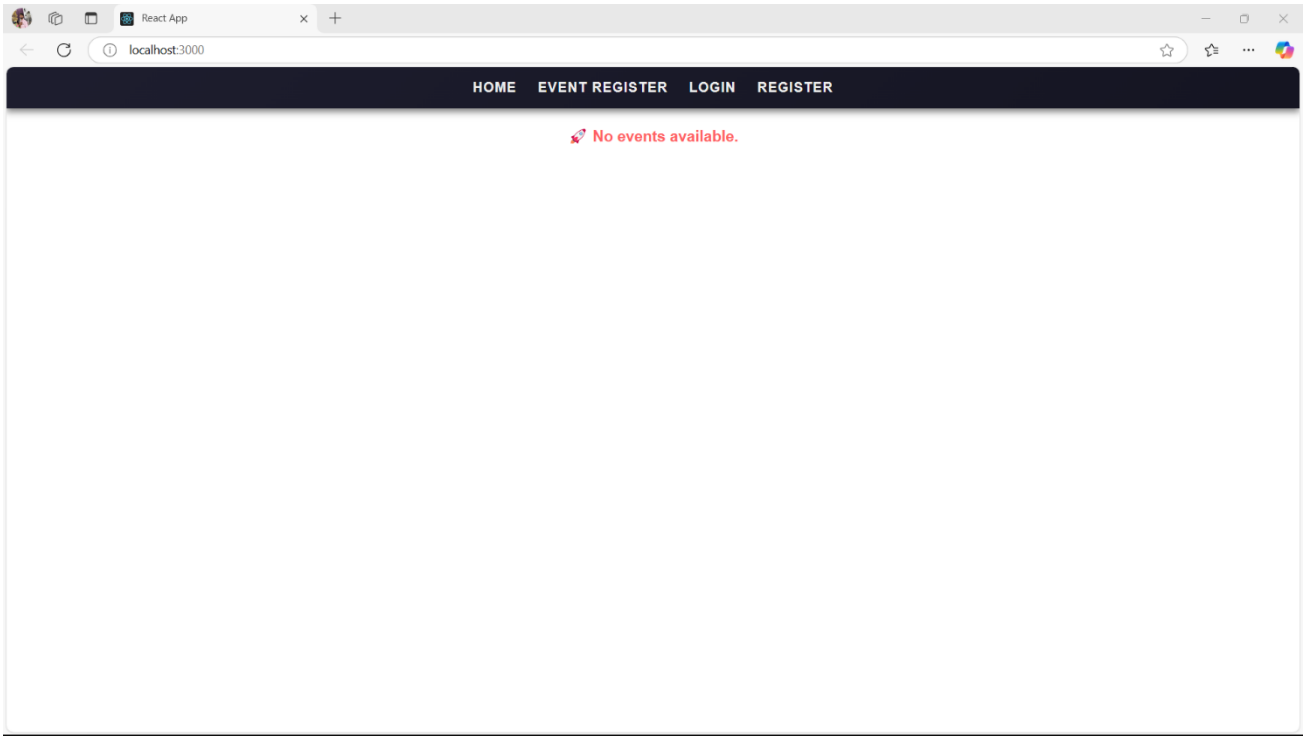


Fig 5.1 Landing Page

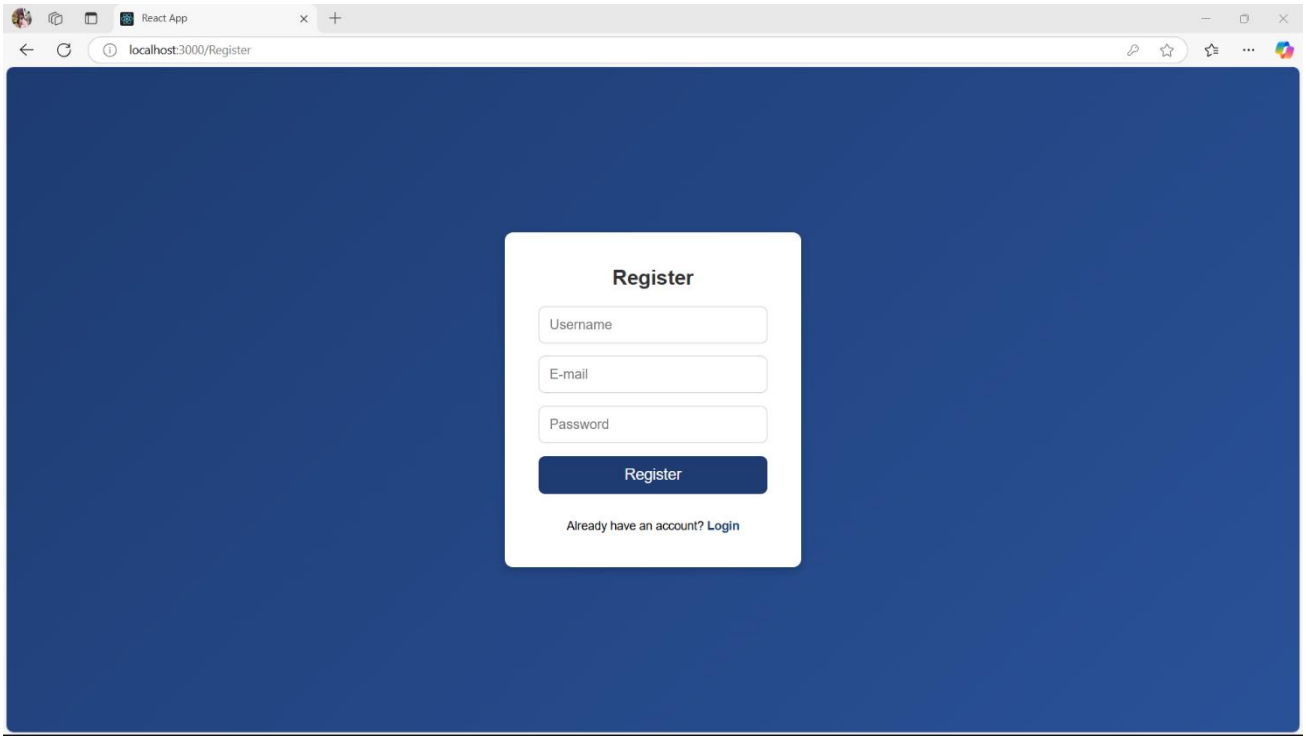


Fig 5.2 Register Page

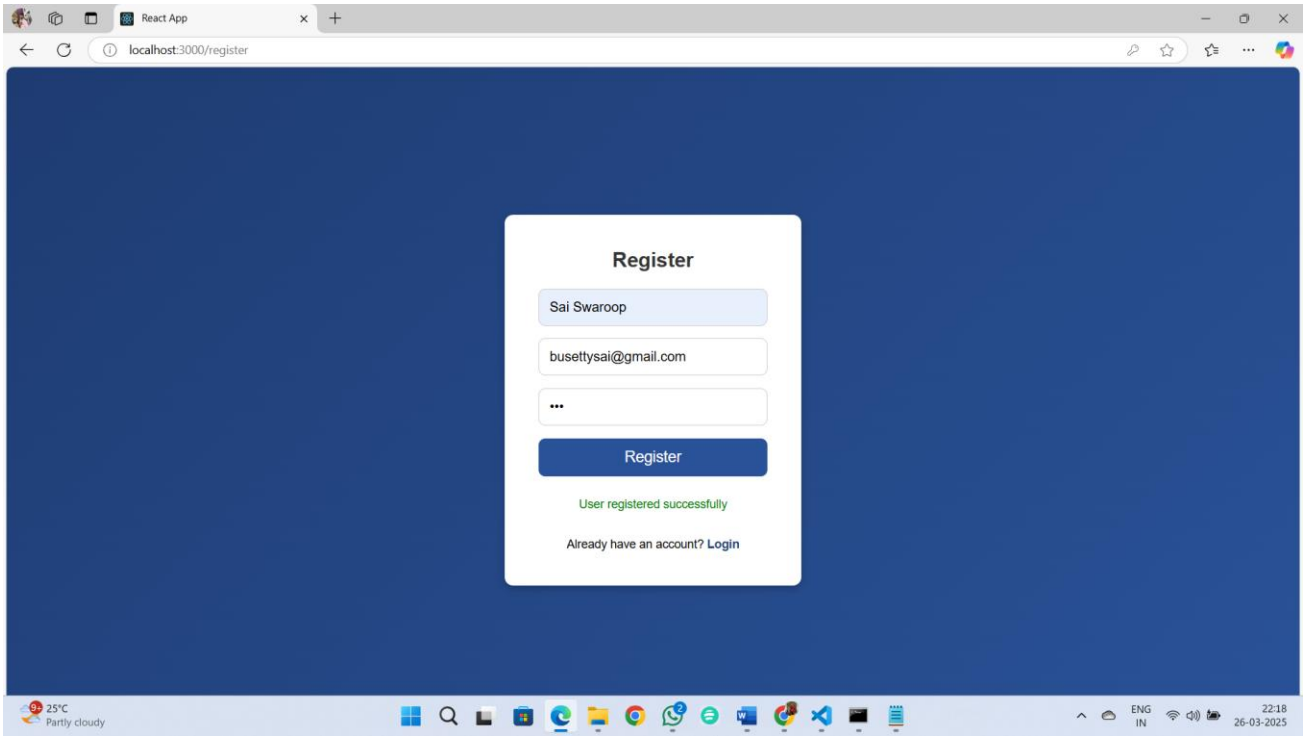


Fig 5.3 After Registration

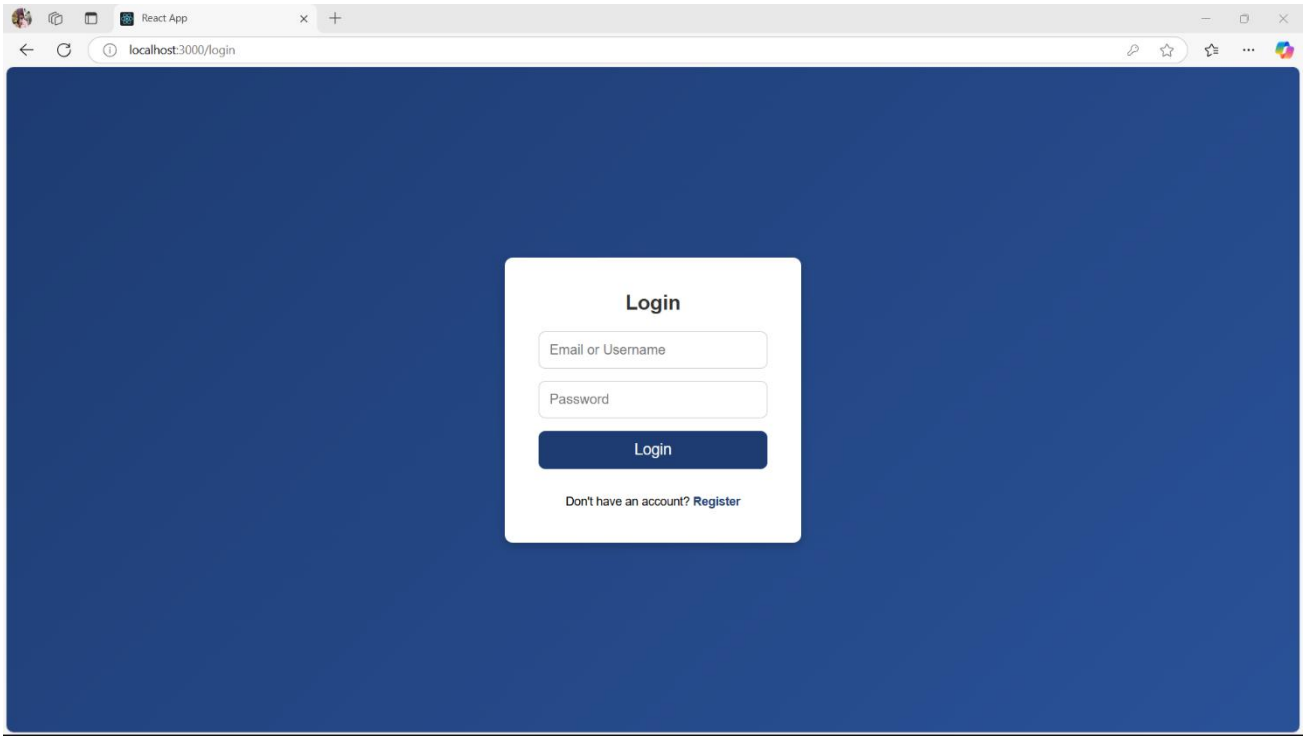


Fig 5.4 Login Page

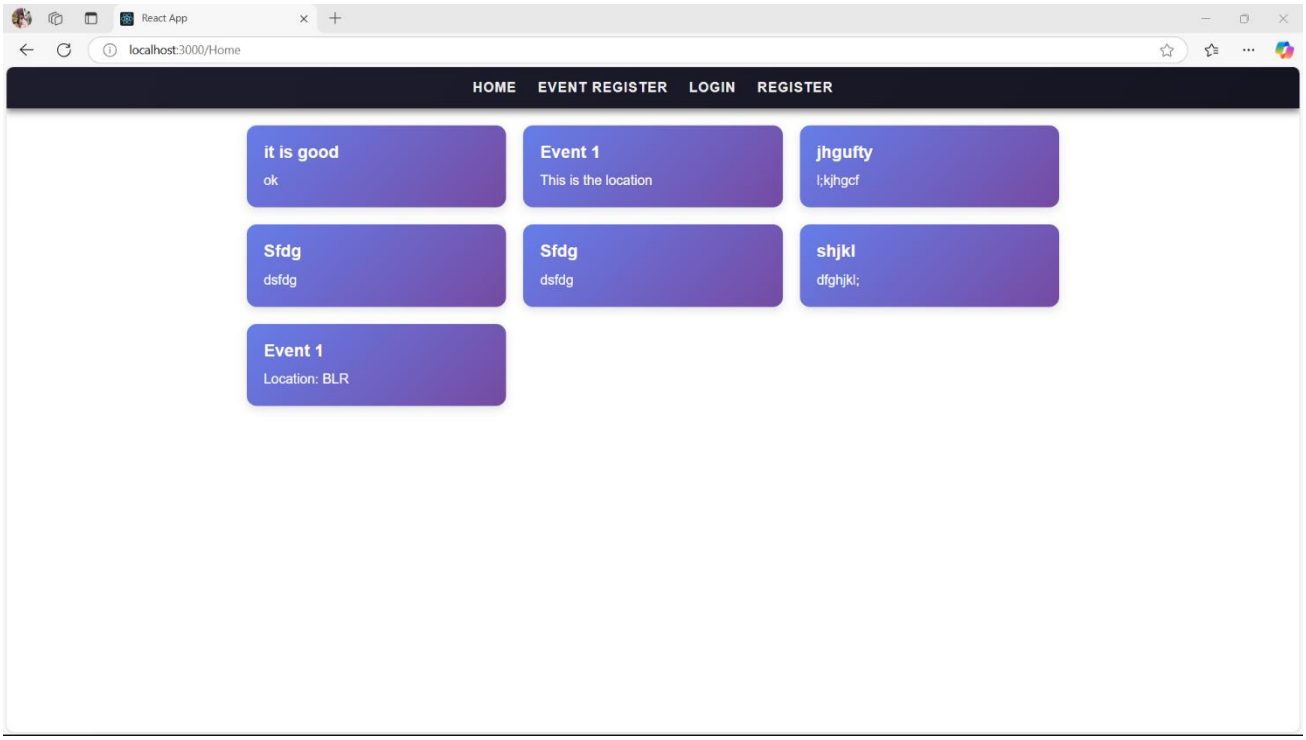


Fig 5.5 Home Page

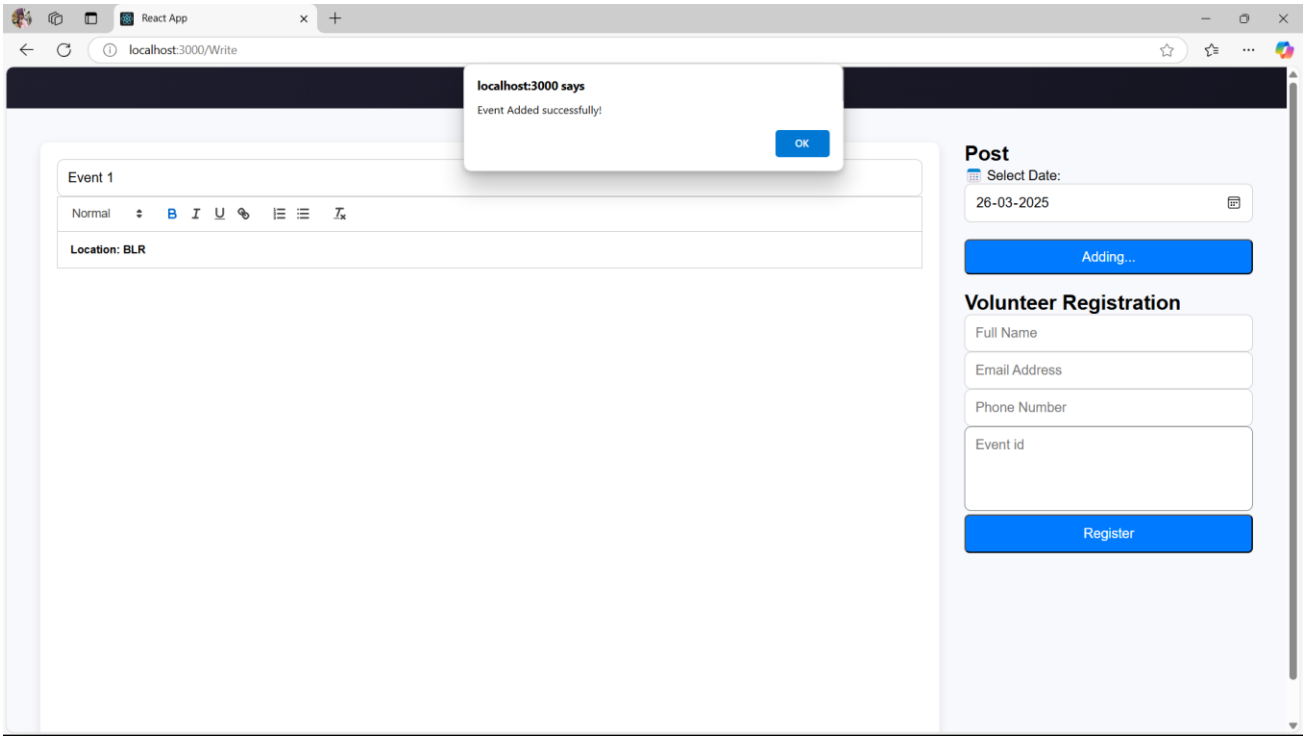


Fig 5.6 After Adding the Event

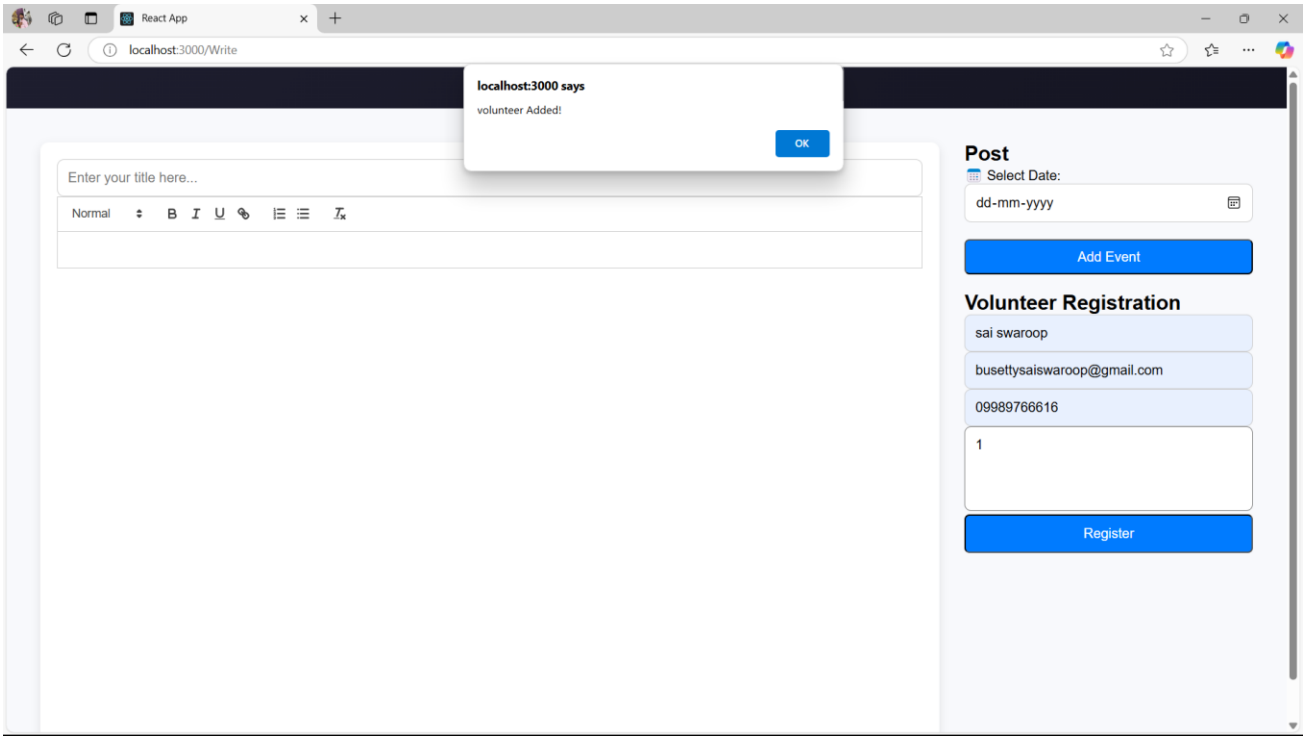


Fig 5.7 Adding the Volunteer

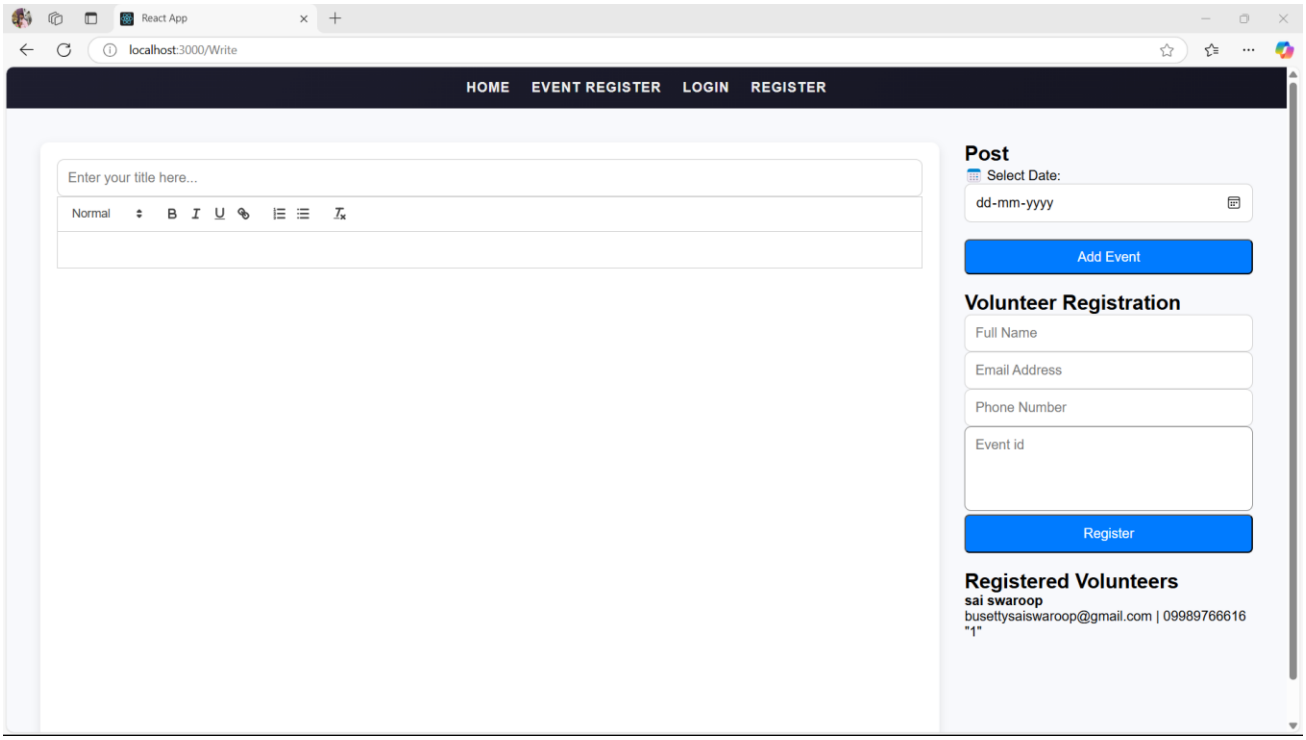


Fig 5.8 After adding the Volunteer

# CONCLUSION

This project successfully developed a dual-portal Volunteer Management Web Application to optimise volunteer coordination. The system provides administrators with comprehensive tools for event and task management, volunteer oversight, and reporting while empowering volunteers to access assigned tasks, event details, and opportunities to participate in self-selected activities. By prioritizing user-friendliness and accessibility, the application streamlines workflows enhances engagement and maximises the impact of volunteer efforts on organisations.

# REFERENCES


- React Documentation: https://react.dev/
- Node.js Documentation: https://nodejs.org/
- Express.js Documentation: https://expressjs.com/
- Sequelize Documentation: https://sequelize.org/
- MySQL Documentation: https://dev.mysql.com/doc/